



1. Что именно мы автоматизируем в code review

Когда я говорю об автоматизации ревью, я не имею в виду автоматическое решение «этот код хороший» или «этот код плохой». Code review состоит из набора проверок, и часть из них хорошо поддаётся автоматизации, а часть по-прежнему требует инженерного суждения.

Автоматизированный reviewer может получить изменения Pull Request, сопоставить их с правилами проекта, найти потенциальные дефекты, проверить обработку ошибок, обратить внимание на изменение публичных контрактов и сформировать комментарии. Но он не знает автоматически всех причин, по которым команда выбрала конкретную архитектуру или бизнес-правило.

Поэтому я рассматриваю AI-review как дополнительный фильтр перед человеческим review. Его задача — быстрее находить конкретные проблемы и давать проверяемые замечания, а не заменять ответственность разработчика или человека, который принимает изменение.

2. Ревью должно анализировать изменение, а не весь код одинаково

Основным объектом автоматического ревью обычно является diff Pull Request. Нас интересует прежде всего то, что изменилось относительно целевой ветки.

Это позволяет сосредоточить внимание на новых рисках. Если разработчик изменил пять строк в сервисе оплаты, нет необходимости одинаково подробно комментировать все несколько тысяч строк проекта.

Но одного diff иногда недостаточно. Изменённая функция может вызывать другой компонент, использовать общий контракт или зависеть от существующей модели данных. Поэтому хороший reviewer начинает с diff, а при необходимости получает дополнительный контекст из репозитория.

В GitHub CLI изменения конкретного PR можно получить через `gh pr diff`. Именно этот механизм используется и в официальном сценарии Hermes для PR-review: агент получает Pull Request, запрашивает его diff, анализирует изменения и только после этого формирует результат.

3. Критерий корректности важнее стилистических замечаний

Первый уровень ревью, который я считаю наиболее важным, — корректность поведения.

Я проверяю, может ли новый код вернуть неправильный результат, нарушить существующий сценарий, неправильно обработать пустые данные или изменить контракт функции. Здесь особенно важны пограничные случаи, потому что основной успешный сценарий разработчик обычно проверяет сам.

Например, изменение может прекрасно работать для одного товара, но неправильно рассчитывать результат для пустой корзины. Или новый обработчик API может возвращать успешный ответ даже после ошибки сохранения данных.

AI-reviewer полезен тем, что может последовательно пройти по изменённым условиям и попытаться найти сценарий, который разработчик не учёл. Но замечание должно быть конкретным: я хочу видеть файл, место изменения, возможный сценарий ошибки и объяснение последствий.

4. Стилистические правила не должны превращать review в шум

Следующий слой — качество и читаемость кода. Здесь можно проверять именование, избыточное дублирование, слишком сложные условия, мёртвый код или неконсистентную обработку ошибок.

Но стилистическое ревью особенно легко автоматизировать неправильно. Если AI начинает комментировать каждую переменную, каждый перенос строки и любое решение, которое можно написать другим способом, Pull Request быстро заполняется десятками малоценных сообщений.

Поэтому часть формальных требований лучше оставлять линтеру и formatter. AI-review должен концентрироваться на тех аспектах читаемости, которые влияют на понимание и сопровождение программы.

В официальном примере code-review Skill для Hermes также рекомендуется не создавать замечания по незначительным стилевым деталям, если они не влияют на читаемость, и не изобретать проблемы только ради того, чтобы оставить комментарий.

5. Архитектурное ревью проверяет границы компонентов

Более сложный уровень автоматического review связан с архитектурой.

Здесь меня интересует не красота отдельной функции, а то, где появилась новая ответственность и не нарушает ли изменение уже существующие границы системы.

Например, frontend-компонент не должен начинать напрямую обращаться к базе данных, а обработчик HTTP-запроса не должен внезапно содержать всю бизнес-логику оплаты. Формально такой код может работать и даже проходить тесты, но архитектура проекта постепенно ухудшается.

Чтобы AI мог находить такие проблемы, архитектурные правила должны быть сформулированы явно. Они могут находиться в документации проекта или отдельном review Skill. Если правила нигде не записаны, модель начинает оценивать архитектуру на основании общих представлений, которые могут не совпадать с решениями команды.

Поэтому автоматизация архитектурного review начинается не с модели, а с формализации стандартов проекта.

6. Безопасность требует отдельного набора критериев

Security review нельзя сводить к поиску слова `password` в изменениях.

Я обращаю внимание на то, какие данные теперь принимает приложение, как они проверяются, какие запросы строятся, изменились ли правила авторизации и не начал ли код раскрывать чувствительные сведения.

Например, новый endpoint может быть корректно реализован технически, но не проверять принадлежность объекта текущему пользователю. Такой дефект относится не к стилю и не к архитектуре, а непосредственно к безопасности.

AI полезен здесь как дополнительный аналитик, но его вывод не означает доказательство отсутствия уязвимостей. Некоторые проблемы требуют специализированного статического анализа, динамического тестирования или проверки специалистом по безопасности.

Кроме того, сам автоматический reviewer работает с недоверенным содержимым Pull Request. Hermes отдельно предупреждает, что название PR, описание и commit-сообщения могут содержать prompt injection, поэтому webhook-based review рекомендуется запускать в изолированной среде.

7. Хороший комментарий review должен быть проверяемым

Автоматический reviewer полезен только тогда, когда его комментарий можно проверить и использовать.

Комментарий вроде «здесь возможна проблема с безопасностью» практически бесполезен. Мне нужно понимать, где находится проблема, при каких условиях она проявляется и что именно следует изменить.

Поэтому результат review желательно связывать с конкретным файлом и строкой изменения. GitHub поддерживает review comments непосредственно на строках diff через API Pull Request review comments. Для такого комментария указываются файл, сторона diff и номер строки.

Внутри комментария можно не только описывать проблему, но и ссылаться на внутренний стандарт проекта. Например, если изменение нарушает правило работы с SQL, reviewer может указать конкретное положение `CONTRIBUTING.md` или другого документа. Тогда замечание становится не личным мнением модели, а проверкой относительно принятого правила.

Если предлагаемое исправление небольшое и однозначное, GitHub review comment также позволяет оформить его как предложение изменения. Но даже такое предложение должно оставаться предложением, а не автоматически применяемой правкой без проверки разработчика.

8. Облачное автоматическое ревью на примере OpenAI Codex

Существует вариант, где инфраструктуру PR-review предоставляет сам облачный AI-сервис.

Например, текущий Codex от OpenAI может быть подключён к GitHub-репозиторию и автоматически выполнять review Pull Request, когда PR переходит из draft в ready. Также review можно запросить непосредственно из PR через обращение к Codex. Сервис сопоставляет заявленное назначение PR с изменениями, анализирует кодовую базу и может запускать код и тесты для проверки выводов.

Такой подход удобен тем, что команде не нужно самостоятельно строить polling, запускать локального агента и писать интеграцию публикации комментариев.

Но для нашей практики я этот путь использовать не буду. Нам нужна схема, которая не зависит от наличия соответствующей облачной подписки и при этом позволяет увидеть, как именно устроена автоматизация. Поэтому мы реализуем reviewer самостоятельно на базе Hermes и GitHub CLI.

9. AI-review можно запускать прямо из GitHub Actions через API-ключ

Другой распространённый вариант заключается в том, что AI-review становится обычным CI job.

В `.github/workflows/*.yaml` можно настроить событие Pull Request. Workflow получает diff, отправляет его во внешний AI API, принимает результат и затем публикует комментарий обратно через GitHub API.

Ключ AI в такой схеме нельзя записывать непосредственно в YAML. Он должен храниться в GitHub Actions Secrets и передаваться job только во время выполнения. GitHub предоставляет repository, organization и environment secrets именно для таких сценариев.

У такого решения есть важная проблема безопасности. Pull Request может содержать недоверенный код. Для PR из fork GitHub по умолчанию не передаёт обычные secrets и ограничивает `GITHUB_TOKEN` чтением. Попытка обойти это через `pull_request_target`, одновременно загрузить код из недоверенного PR и выполнить его с доступом к секретам создаёт известный класс уязвимостей, который GitHub называет pwn request.

Технически вариант с AI API в workflow вполне реален, но в нашем курсе он тоже не подходит: у нас нет внешнего платного API-ключа, а сама цель занятия заключается в том, чтобы автоматизировать review с уже имеющимся Hermes.

10. Какой вариант автоматизации мы будем реализовывать

Для нашей практики я выбираю схему Hermes + GitHub CLI + периодический polling.

Hermes официально описывает именно такой вариант PR-review agent. Gateway запускает cron-задачу по расписанию, агент через `gh pr list` находит открытые или недавно обновлённые Pull Request, через `gh pr diff` получает изменения, применяет постоянный `code-review` Skill и затем может опубликовать результат обратно в GitHub. Такой вариант не требует публичного webhook endpoint и работает даже за NAT или firewall.

Для общего review GitHub CLI предоставляет `gh pr review`: агент может оставить комментарий, запросить изменения или оформить approve. Для обычного комментария также доступен `gh pr comment`.

Если нам потребуется комментарий непосредственно к определённой строке diff, мы сможем дополнить эту схему вызовом GitHub REST API через `gh api`. GitHub API поддерживает создание line-level review comments с указанием файла и строки, поэтому технически наша локальная Hermes-система способна публиковать не только общий отчёт, но и точечные замечания.

Важная часть будущей практики будет находиться не в самом запуске модели, а в `code-review` Skill. Именно там мы зафиксируем критерии корректности, безопасности и качества, формат замечаний и правило ссылаться на стандарты проекта. После этого Hermes сможет использовать ту модель, которая уже доступна в нашей конфигурации, включая бесплатный или локальный вариант, а GitHub будет отвечать только за Pull Request и публикацию результатов.

Таким образом, автоматическое review будет состоять из трёх независимых частей: GitHub предоставляет изменения, Hermes выполняет интеллектуальный анализ по заранее заданным критериям, а GitHub CLI возвращает результат в Pull Request. Мы не привязываем учебную архитектуру ни к подписке Codex Cloud, ни к OpenAI API key внутри GitHub Actions.
